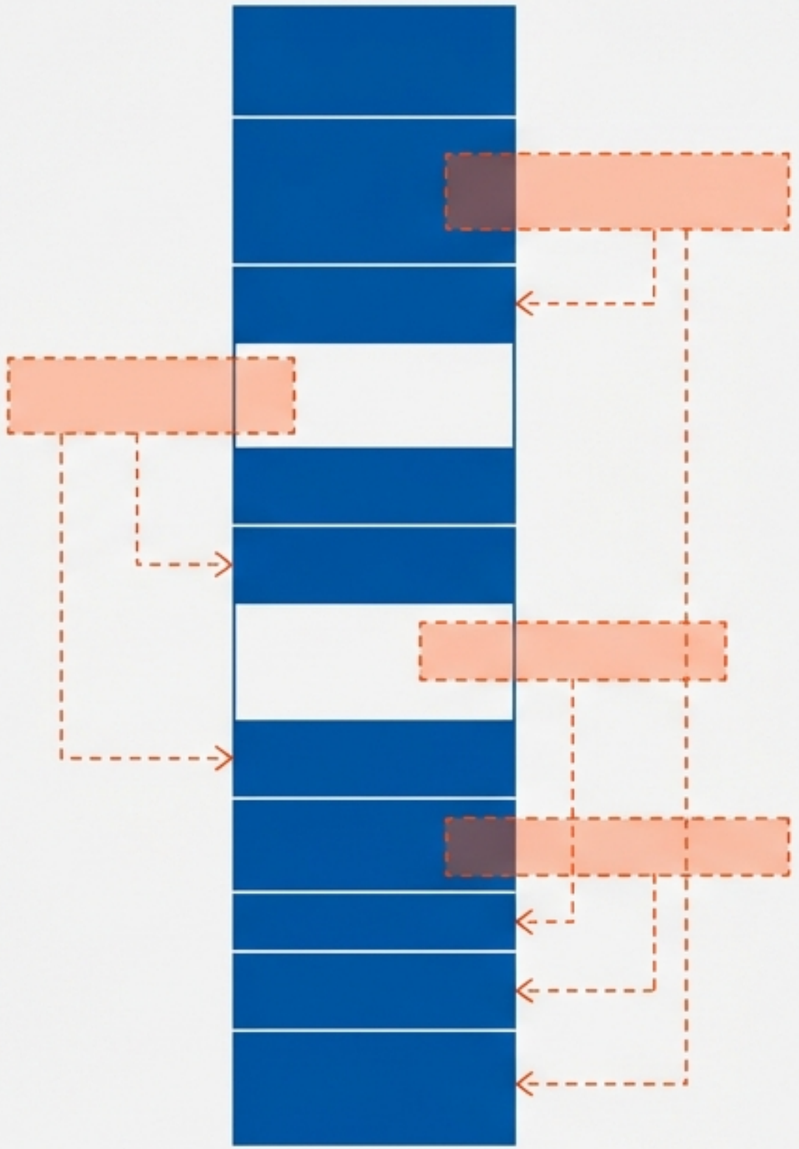
Advanced Univariate Imputation.

Random Sampling, Missing Indicators & Automated Parameter Selection



HANDLING MISSING DATA: PART 4

NotebookLM

THE IMPUTATION LANDSCAPE

ALERT: MISSING DATA SIGNAL

STAGE 1

Complete Case Analysis (Deletion)

Removes any row with missing data. High risk of bias.

STAGE 2

Basic Imputation (Mean/Median/Mode)

Fills gaps with central tendency. Distorts distribution and variance.

STAGE 3

Advanced Univariate (Random Sample & Indicators)

Preserves variance by sampling from existing values. Tracks missingness with indicator flags.

STAGE 4

Multivariate (KNN, MICE)

Models missing values based on other features. Complex, computationally intensive.

THE CONTEXT

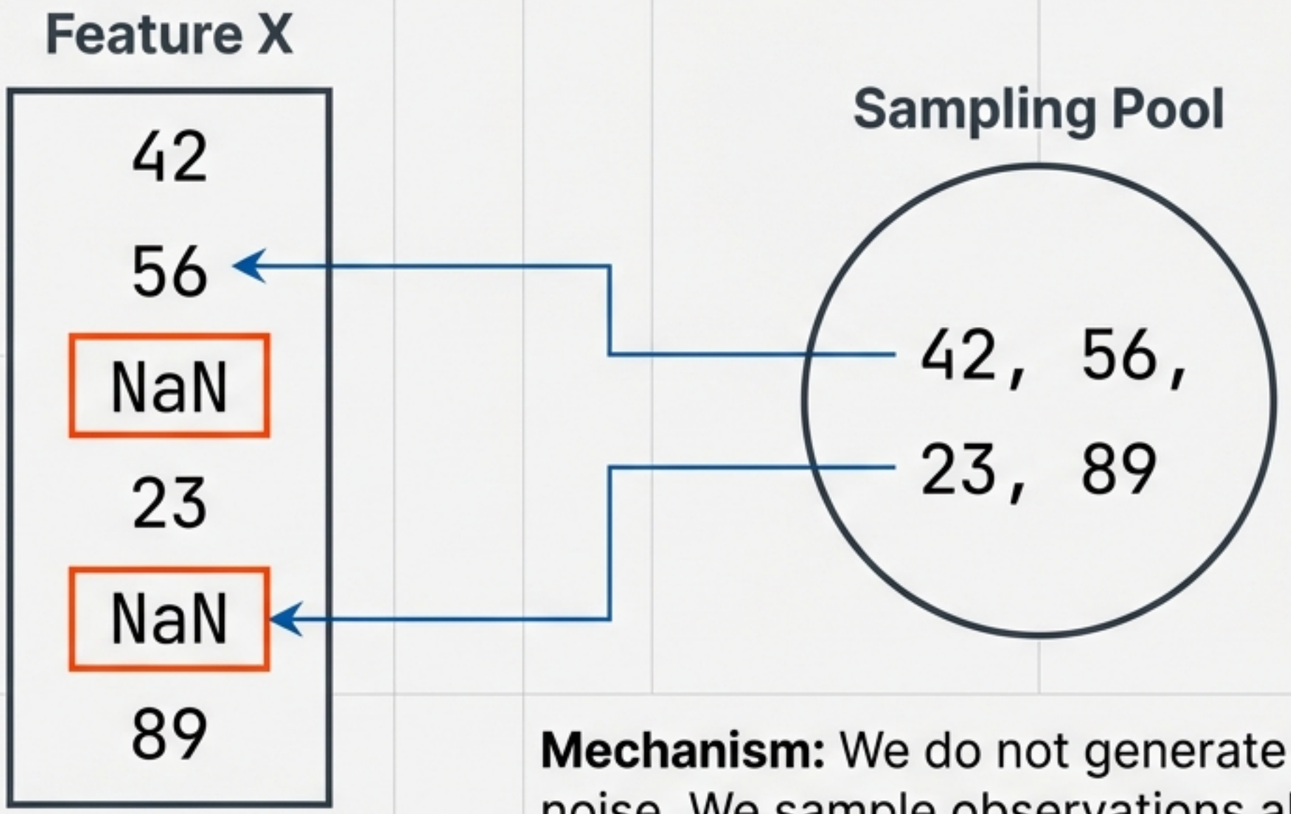
Standard methods like Mean/Median fill gaps but distort the data's natural shape. We need techniques that respect the distribution and relationships within the data.

THIS MODULE

1. Random Sample Imputation (Preserving Variance).
2. Missing Indicators (Capturing Signal).
3. **GridSearchCV** (Automating Decisions).

NotebookLM

Concept: Random Sample Imputation



Mechanism: We do not generate random noise. We sample observations already present in the training set to fill the gaps.

Logic:

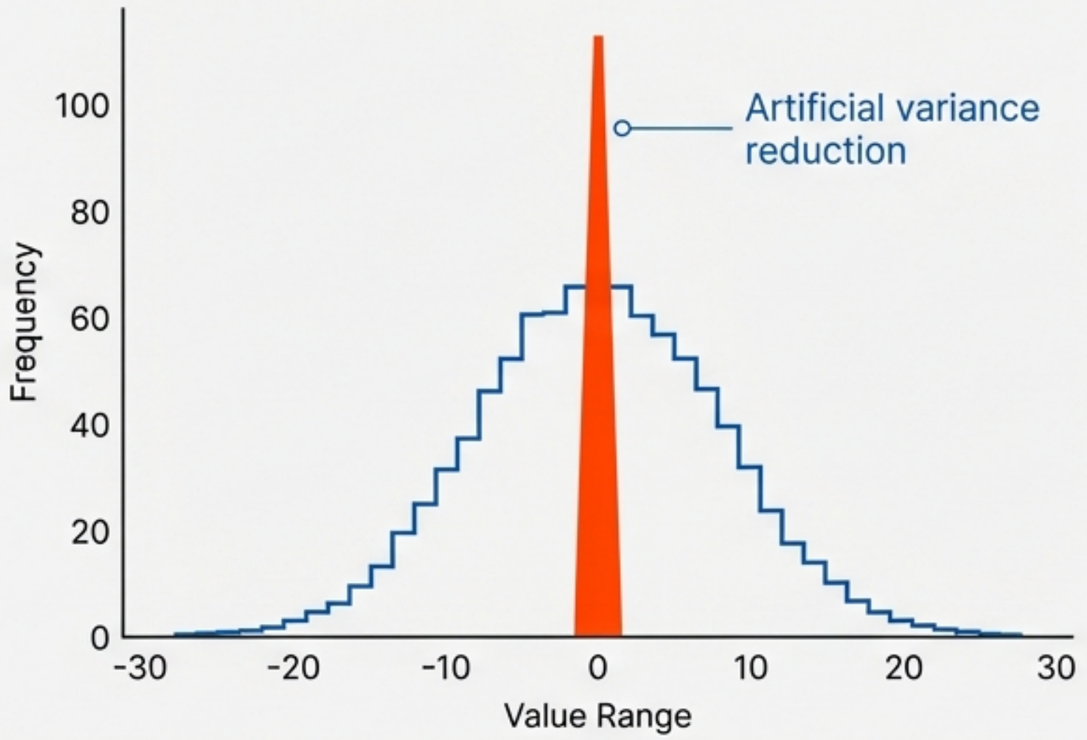
If a value appears frequently in the data (e.g., Age 30), it has a higher probability of being selected to fill a gap.

NotebookLM

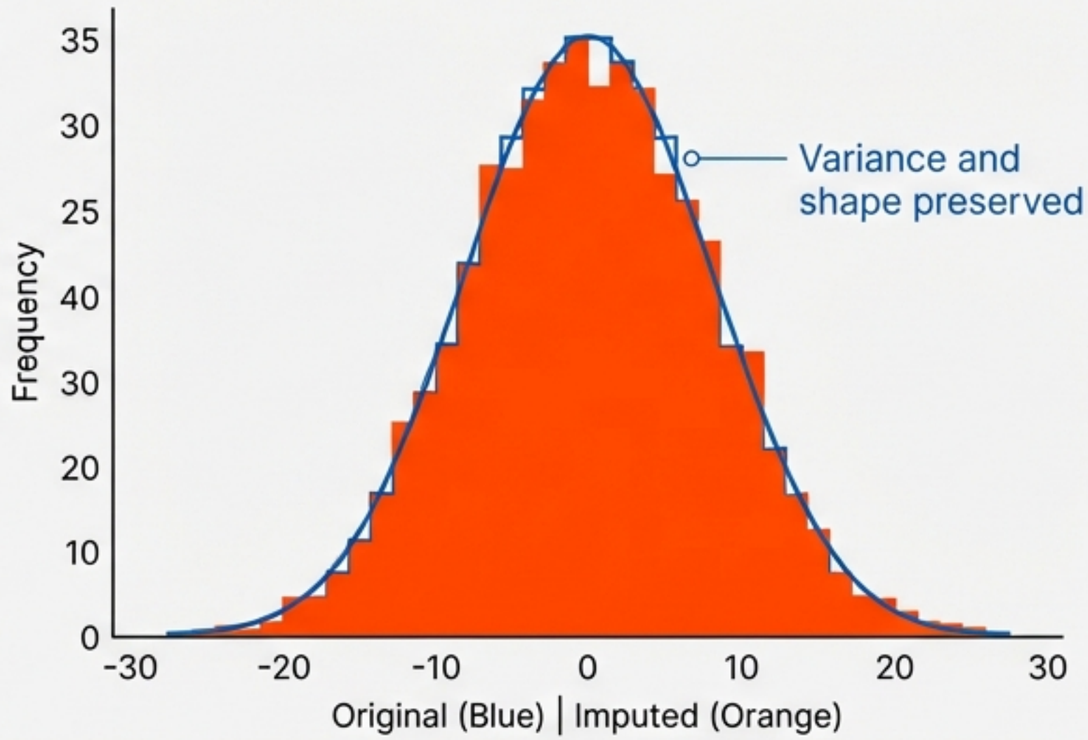
PRESERVING DISTRIBUTION SHAPE

Why Random Sampling outperforms Mean Imputation for Linear Models

MEAN IMPUTATION



RANDOM SAMPLE IMPUTATION

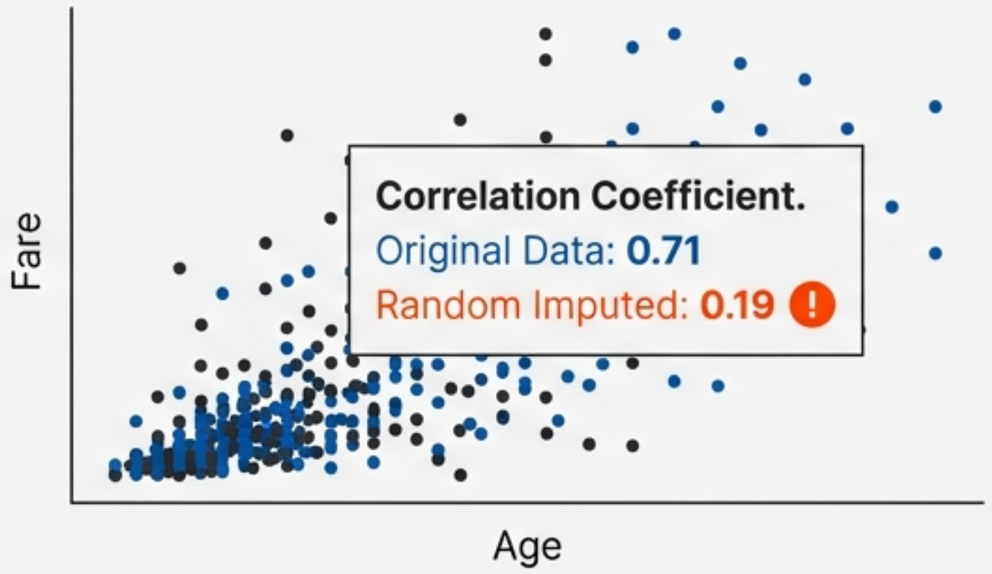


Ideal for Linear Regression and Logistic Regression where distribution normality assumptions are critical

NotebookLM

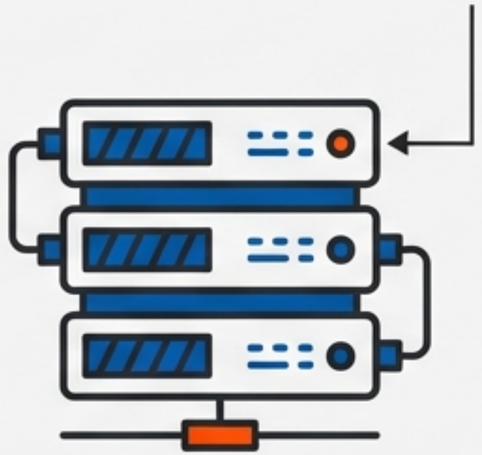
The Costs: Covariance & Memory.

The Covariance Problem.



Injecting random values destroys the relationship between the imputed column and other features.

The Memory Problem.

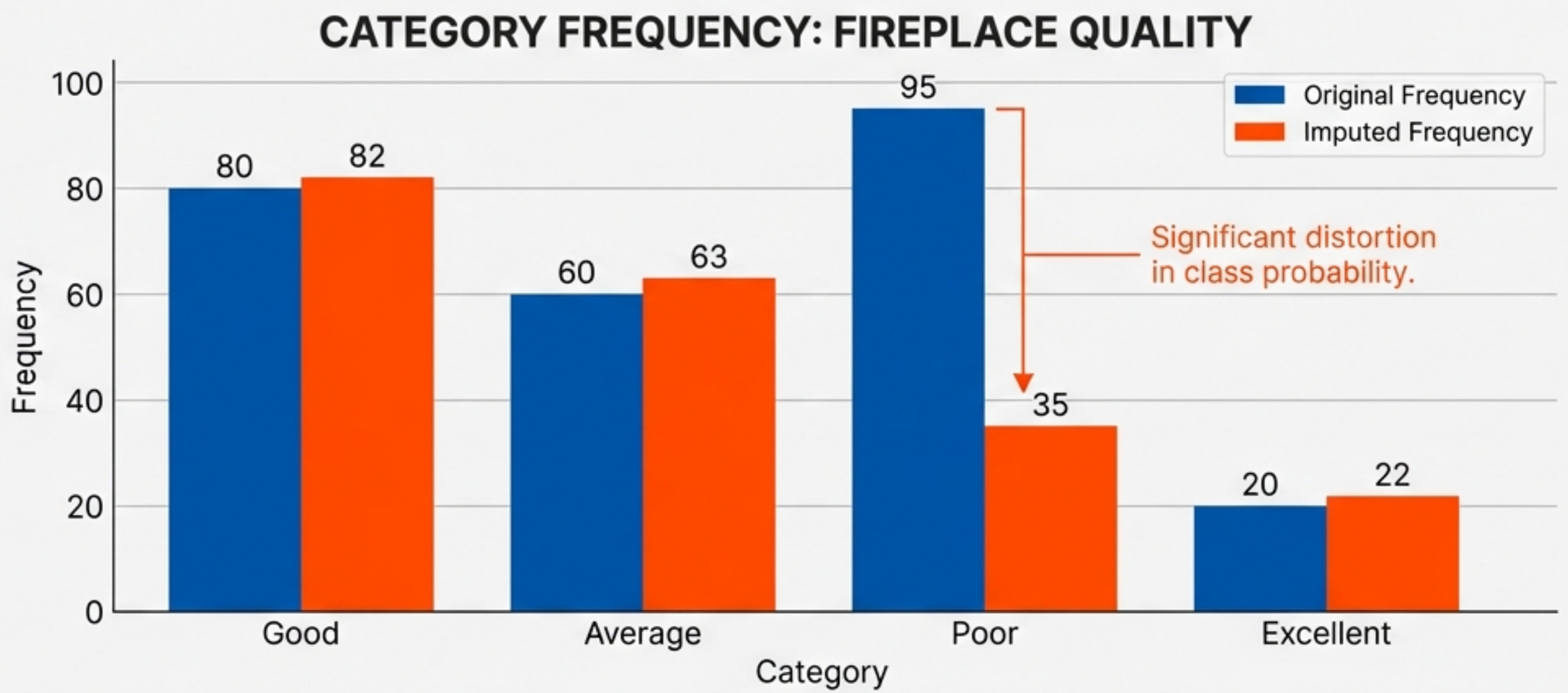


To deploy this model, the *entire* training column must be stored in memory to sample from during inference. Heavy computational cost for large datasets.

NotebookLM

CATEGORICAL DATA & RISKS

The “Red Flag” Scenario

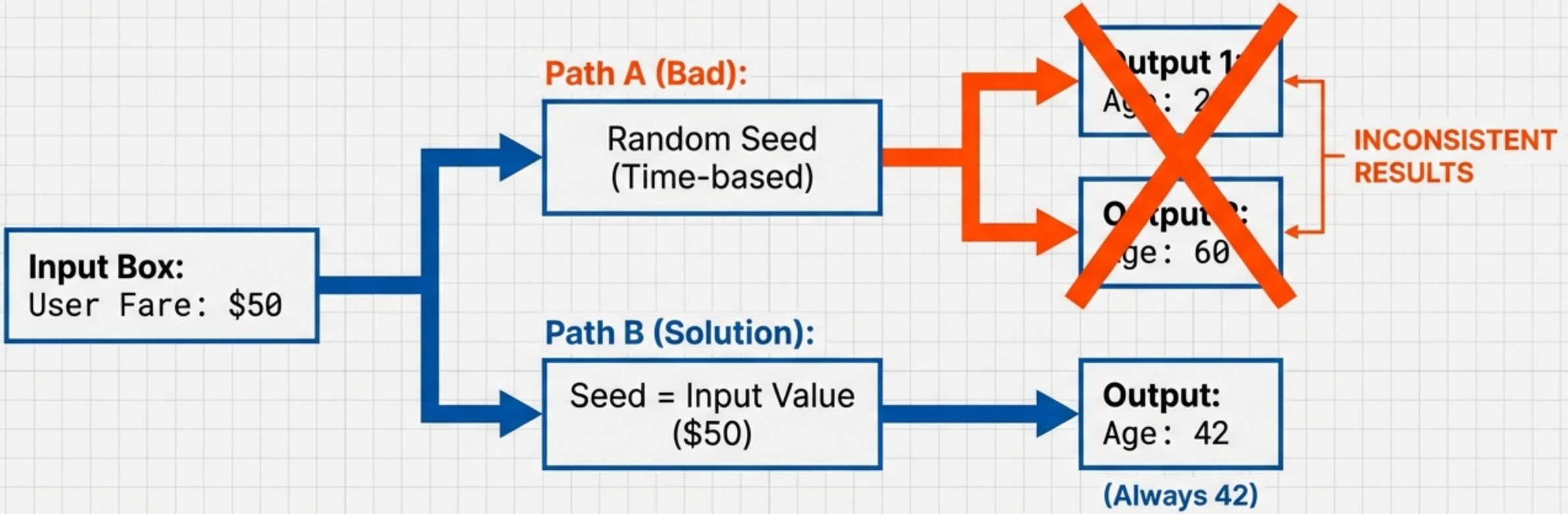


Warning: If the missing data percentage is high (e.g., >50%), random sampling can fundamentally alter the probability of specific categories appearing. Always verify class ratios before and after.

NotebookLM

Production Engineering: Controlled Randomness.

In a live application, identical inputs must yield identical predictions.
Pure randomness creates poor UX.



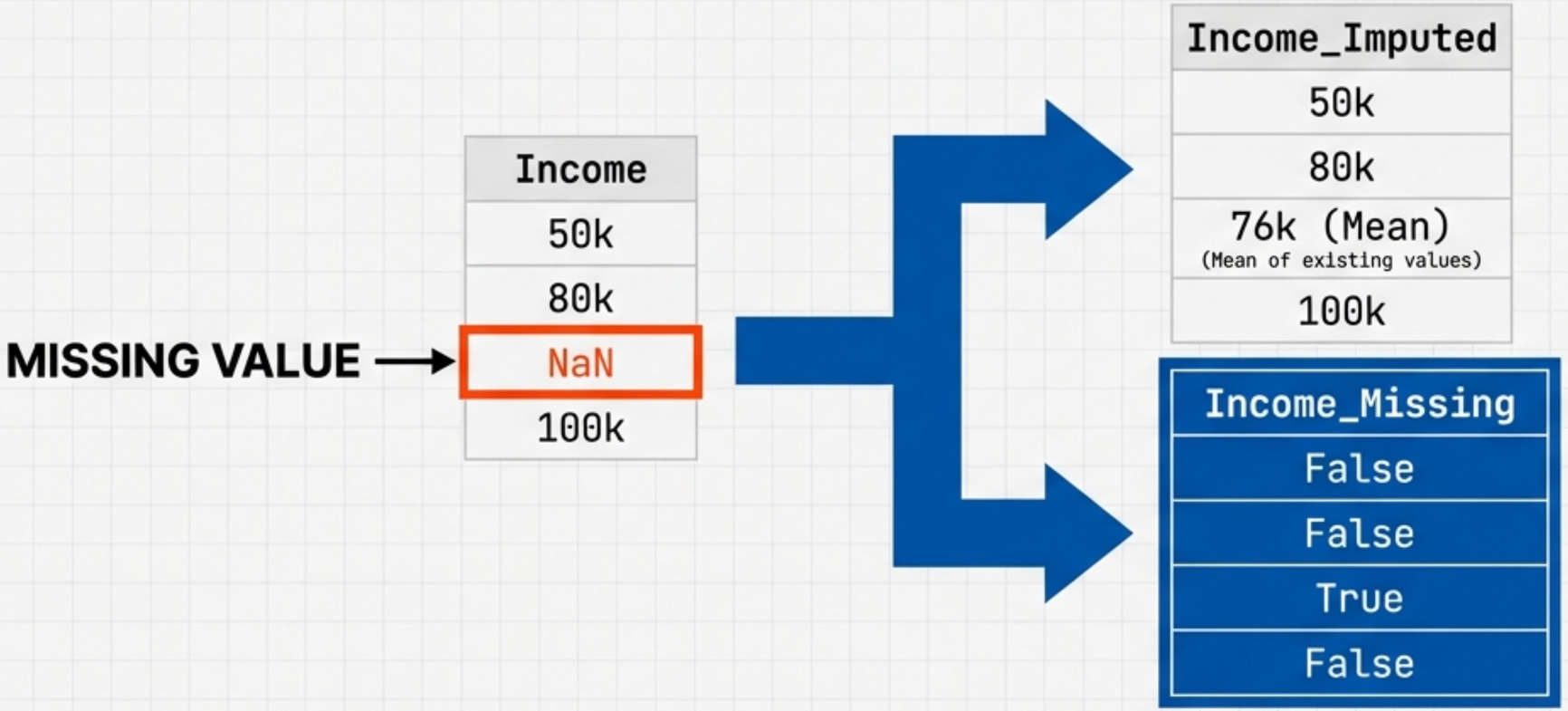

```
X_train['Age'].sample(1, random_state=int(observation['Fare']))
```

By tying the random state to the input variable, we ensure deterministic randomness.

NotebookLM

THE MISSING INDICATOR

Finding Signal in the Noise



Sometimes, the fact that data is missing is information in itself. By explicitly telling the model “This value was faked”, we allow it to treat imputed observations differently.

Insight derived from the KDD Cup, where this boolean flag significantly improved model accuracy.

NotebookLM

Implementation in Scikit-Learn.

Method A: Manual.

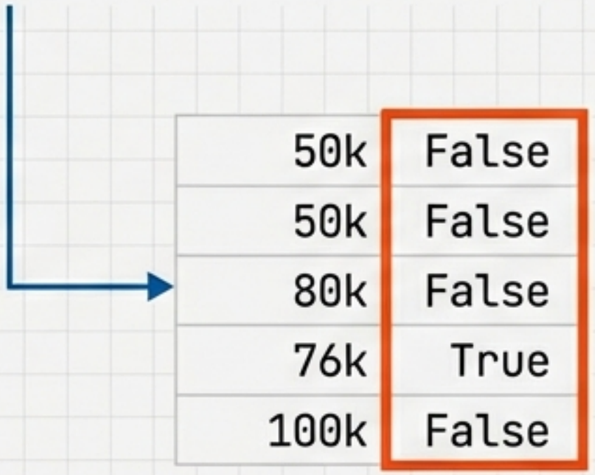
1. Use MissingIndicator class.
2. Create boolean matrix.
3. Concatenate with original data.

Status: Cumbersome.

Method B: Integrated.

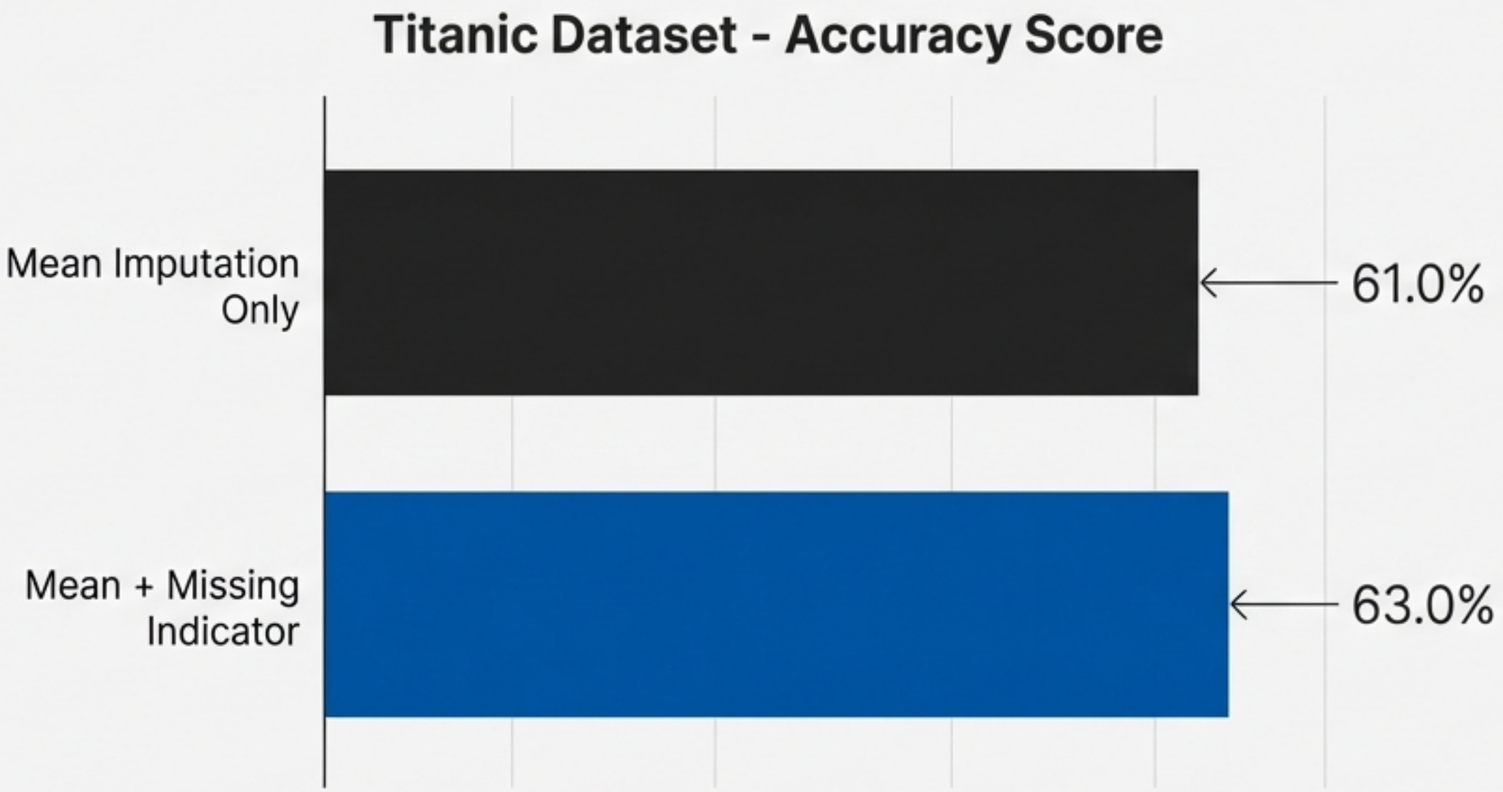
```
imputer = SimpleImputer(strategy='mean',
                        add_indicator=True)
X_transformed = imputer.fit_transform(X)
```

The ‘add_indicator=True’ parameter automatically appends the boolean mask to the output array. **Efficient and pipeline-ready.**



NotebookLM

Performance Reality Check



ANALYSIS

The Verdict: It is not a magic bullet. In this case, it provided a marginal **+2%** lift.

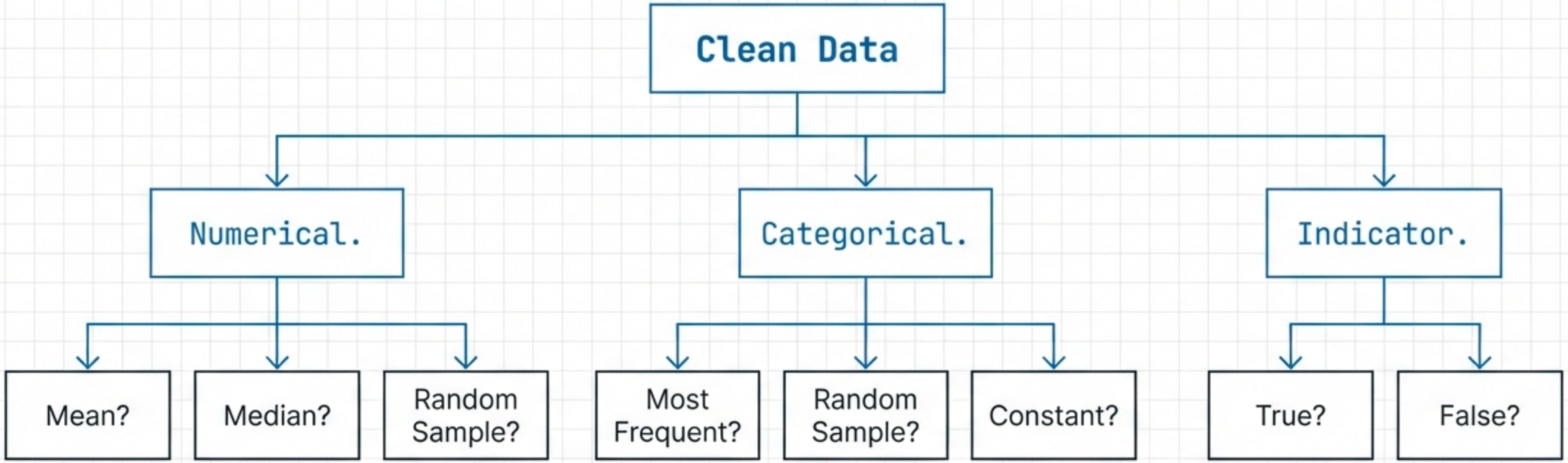
The Cost: Increased Dimensionality. For every column with missing data, you add a new feature.

STRATEGY

Use when ‘missingness’ is systemic (MNAR), not random (MCAR). It is a hyperparameter to be tuned, not a default setting.

NotebookLM

The Selection Dilemma

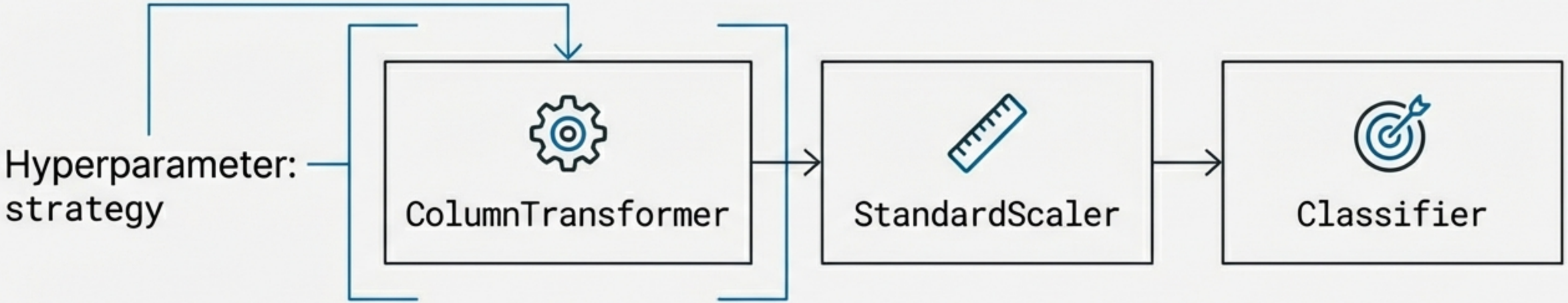


Which combination is optimal?

Manual trial and error for every permutation is inefficient. We need to treat ‘Imputation Strategy’ as a hyperparameter to be optimized.

NotebookLM

Solution: GridSearch CV.



Instead of hard-coding mauilized strategies, to echage CIV atar and eamplictne rleating, when there, is hyperparameter: strategy in modural block.

Instead of hard-coding 'mean', we pass a list of `strategies ['mean', 'median', 'most_frequent']` to the GridSearch.

The machine iterates through the pipeline, training the model on every variation, and returns the mathematically optimal cleaning method.

NotebookLM

The Automation Code

```
param_grid = {
    'preprocessor__num__imputer__strategy': ['mean', 'median'],
    'preprocessor__cat__imputer__strategy': ['most_frequent', 'constant'],
    'classifier__C': [0.1, 1.0, 10, 100]
}

grid_search = GridSearchCV(pipeline, param_grid, cv=5)
grid_search.fit(X_train, y_train)
```

Tuning the Numerical Cleaning Method.

Tuning the Categorical Cleaning Method.

Result: `grid_search.best_params_` tells you exactly how to clean this specific dataset.

NotebookLM

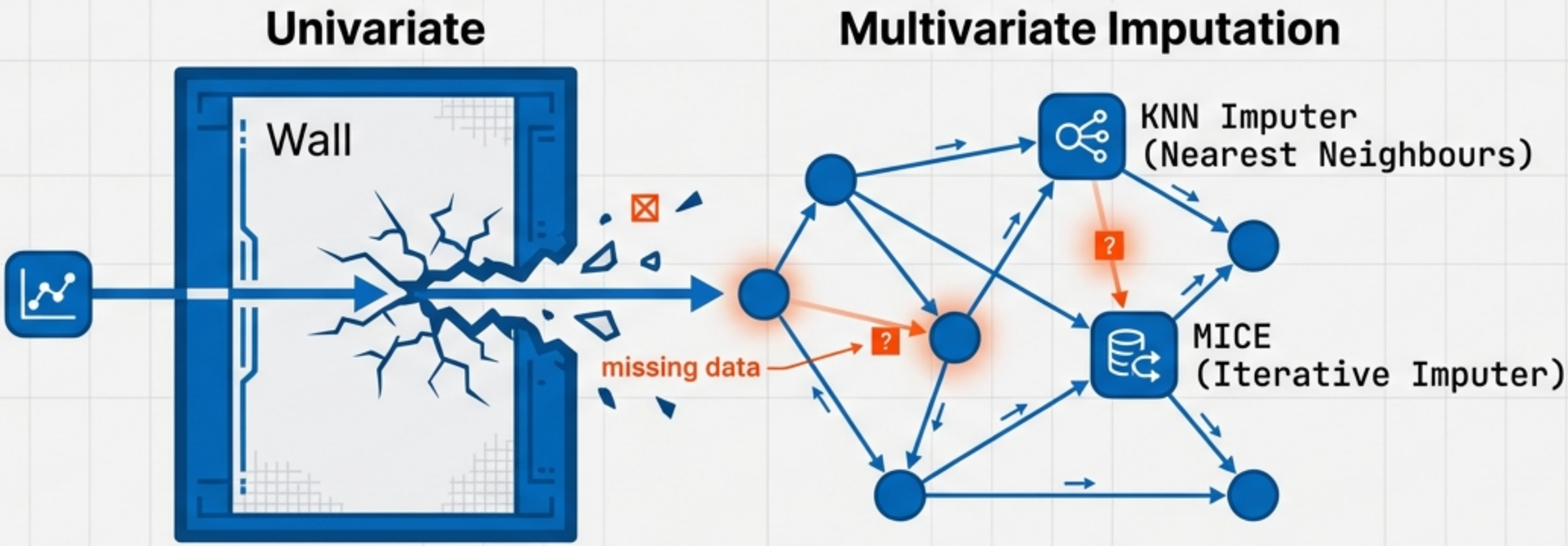
Technique Comparison Matrix.

Method	Best For	Trade-off
Random Sample Imputation	Linear Models (Preserves Distribution)	Heavy memory usage; Breaks correlations.
Missing Indicator	Non-Random Missingness (Signal)	Increases dimensionality.
GridSearch Automation	Unbiased Selection	Computationally expensive training.

NotebookLM

Beyond Univariate Imputation

We have mastered cleaning columns in isolation (Univariate). But data features are rarely independent. The final frontier uses the relationships *between* columns to predict missing values.



Coming Next: Part 5 & 6.

NotebookLM